

Contents

Les flux itératifs	3
1 Présentation des flux itératifs	3
2 La boucle while	4
2.1 Principe général	4
2.2 Fonctionnement détaillé	4
2.3 Boucle infinie maîtrisée	5
2.4 Validation d'entrée robuste	5
3 La boucle for	6
3.1 Principe général	6
3.2 Parcours d'une liste	6
3.3 Utilisation de range	6
3.4 enumerate	7
3.5 zip — itération parallèle	7
Cas avancé — zip avec dict	7
3.6 Parcours d'un dictionnaire	8
4 break et continue	8
4.1 break	8
4.2 continue	8
5 Clause else des boucles	9
6 any() et all()	9
7 Compréhensions de listes — aperçu	10
8 Cas métier — Gestion de commandes	10
9 Imbrication contrôlée	11
Pièges fréquents	11
Modifier une liste pendant le parcours	11
Variable de boucle accessible après la boucle	12
1 1 Comparaison while vs for	12
1 2 Ouverture — iter() et next()	12
1 3 Mini-exercices de production	13
Exercice 1 — Prédiction	13
Exercice 2 — Production	13
Exercice 3 — Débogage	14

Synthèse	15
20 Questions Intelligentes — Flux Itératifs	15
1) Différence fondamentale entre while et for	15
Énoncé	15
Solution	15
2) Pourquoi une boucle while peut devenir infinie ?	16
Énoncé	16
Solution	16
3) break interrompt quelle boucle ?	16
Énoncé	16
Solution	16
4) continue saute quoi exactement ?	17
Énoncé	17
Solution	17
5) Clause else d'une boucle for	17
Énoncé	17
Solution	17
6) Pourquoi enumerate est préférable ?	18
Énoncé	18
Solution	18
7) Modification d'une liste pendant itération	19
Énoncé	19
Solution	19
8) range stop exclusif	19
Énoncé	19
Solution	19
9) range avec pas négatif	20
Énoncé	20
Solution	20
10) zip pour itérer en parallèle	20
Énoncé	20
Solution	20
11) Court-circuit dans while	21
Énoncé	21
Solution	21

12) Boucle sur dictionnaire	21
Énoncé	21
Solution	21
13) Variable de boucle hors scope	22
Énoncé	22
Solution	22
14) Utilité du else dans recherche	22
Énoncé	22
Solution	22
15) while vs input	23
Énoncé	23
Solution	23
16) Double boucle et complexité	24
Énoncé	24
Solution	24
17) Utilisation de any()	24
Énoncé	24
Solution	24
18) Utilisation de all()	25
Énoncé	25
Solution	25
19) Performance : range vs liste	25
Énoncé	25
Solution	26
20) Refactorisation d'une boucle complexe	26
Énoncé	26
Solution	26

Les flux itératifs

Le support est propriété de **Bechir Bejaoui**

1 Présentation des flux itératifs

Les flux itératifs permettent d'exécuter un bloc d'instructions plusieurs fois.

Ils répondent à trois besoins fondamentaux :

- Répéter une action un nombre déterminé de fois
- Répéter une action tant qu'une condition reste vraie
- Parcourir une structure de données

Python propose deux structures principales :

- while
- for

2 La boucle while

2.1 Principe général

La boucle while exécute un bloc tant qu'une condition booléenne est vraie.

Structure :

```
while condition:
    bloc
```

La condition est évaluée avant chaque itération.

2.2 Fonctionnement détaillé

```
"""
Objectif :
Afficher les nombres de 1 à 5.
Montrer le mécanisme complet : initialisation, condition, mise à jour.
"""

i = 1                # Initialisation du compteur

while i <= 5:        # Condition testée avant chaque itération
    print("Valeur actuelle :", i)
    i += 1           # Mise à jour indispensable pour éviter boucle infinie

print("Fin de boucle")
```

Points essentiels :

- Initialisation obligatoire
- Mise à jour obligatoire
- Risque de boucle infinie si oubli

2.3 Boucle infinie maîtrisée

```
"""
Boucle infinie contrôlée via break.
Utilisation typique : menus interactifs, serveurs, processus d'attente.
"""

while True:
    reponse = input("Tapez 'stop' pour quitter : ")

    if reponse == "stop":
        print("Arrêt demandé")
        break

print("Programme terminé")
```

2.4 Validation d'entrée robuste

```
"""
Objectif :
Demander un âge valide (>= 0).
Répéter tant que la saisie est incorrecte.
Concepts mobilisés : while True, continue, break, gestion d'erreur.
"""

while True:
    try:
        age = int(input("Entrez votre âge : "))

        if age < 0:
            print("Âge invalide")
            continue

        break    # sortie si valide

    except ValueError:
        print("Veuillez entrer un nombre valide")

print("Âge enregistré :", age)
```

3 La boucle for

3.1 Principe général

La boucle for parcourt un itérable.

Structure :

```
for element in iterable:
    bloc
```

3.2 Parcours d'une liste

```
"""
Parcours simple d'une collection.
À chaque itération, fruit contient l'élément courant.
"""
```

```
fruits = ["pomme", "banane", "fraise"]
```

```
for fruit in fruits:
    print("Fruit :", fruit)
```

3.3 Utilisation de range

range génère une séquence de nombres sans créer une liste en mémoire.

```
"""
Affichage de 0 à 4.
range(stop) → 0 à stop-1
range(start, stop) → start à stop-1
range(start, stop, step) → avec pas
"""
```

```
for i in range(5):
    print(i)
```

```
for i in range(2, 6):
    print(i)
```

```
for i in range(0, 10, 2):
    print(i)
```

```
# Compte à rebours
```

```
for i in range(5, 0, -1):
    print(i)
```

3.4 enumerate

Obtenir index + valeur simultanément.

```
"""
enumerate évite la gestion manuelle d'un compteur.
Le paramètre start définit la valeur de départ de l'index.
"""
```

```
noms = ["Alice", "Bob", "Charlie"]

for index, nom in enumerate(noms, start=1):
    print(index, "-", nom)
```

3.5 zip — itération parallèle

Parcourir plusieurs itérables simultanément.

```
"""
zip associe les éléments de même position.
S'arrête au plus court des itérables.
"""
```

```
noms = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 22]

for nom, age in zip(noms, ages):
    print(f"{nom} a {age} ans")
```

Cas avancé — zip avec dict

```
"""
Construire un dictionnaire depuis deux listes.
"""
```

```
cles = ["nom", "age", "ville"]
valeurs = ["Alice", 30, "Paris"]

profil = dict(zip(cles, valeurs))
print(profil)
```

3.6 Parcours d'un dictionnaire

```
"""
Parcours clé-valeur avec items().
for d parcourt uniquement les clés.
for d.items() fournit clé + valeur.
"""

utilisateur = {
    "nom" : "Alice",
    "age" : 30,
    "ville": "Paris"
}

# Clés uniquement
for cle in utilisateur:
    print("Clé :", cle)

# Clés et valeurs
for cle, valeur in utilisateur.items():
    print(cle, ":", valeur)
```

4 break et continue

4.1 break

Interrompt immédiatement la boucle courante.

```
"""
Arrêt dès qu'on trouve la valeur cherchée.
break interrompt uniquement la boucle la plus interne.
"""

nombres = [1, 4, 7, 10, 15]

for n in nombres:
    if n == 10:
        print("Trouvé :", n)
        break
```

4.2 continue

Passes directement à l'itération suivante.


```

"""
Ignorer les nombres pairs.
continue ignore le reste du bloc courant.
"""

for i in range(1, 11):
    if i % 2 == 0:
        continue

    print("Impair :", i)

```

5 Clause else des boucles

Le else s'exécute uniquement si la boucle se termine sans break.

```

"""
Recherche avec clause else.
Le else évite l'usage d'une variable drapeau externe.
"""

liste = [2, 4, 6, 8]
recherche = 5

for n in liste:
    if n == recherche:
        print("Trouvé")
        break
else:
    print("Non trouvé")

```

6 any() et all()

Alternatives concises aux boucles de vérification.

```

"""
any() retourne True si au moins un élément est vrai.
all() retourne True si tous les éléments sont vrais.
Plus expressifs qu'une boucle for avec drapeau.
"""

notes = [12, 15, 8, 17]
scores = [11, 13, 14]

```

```

# Au moins une note insuffisante ?
if any(n < 10 for n in notes):
    print("Au moins une note insuffisante")

# Tous les scores valides ?
if all(s >= 10 for s in scores):
    print("Tous les scores sont valides")

```

7 Compréhensions de listes — aperçu

```

"""
Les compréhensions sont la forme idiomatique des boucles for.
Elles seront approfondies en Partie II.
"""

# Version boucle classique
carres = []
for x in range(1, 6):
    carres.append(x**2)

# Version compréhension
carres = [x**2 for x in range(1, 6)]

print(carres)

```

8 Cas métier — Gestion de commandes

```

"""
Parcourir une liste de commandes.
Arrêter si une commande est invalide.
"""

commandes = [
    {"id": 1, "valide": True},
    {"id": 2, "valide": True},
    {"id": 3, "valide": False},
    {"id": 4, "valide": True},
]

for commande in commandes:

```

```

if not commande["valide"]:
    print("Commande invalide détectée :", commande["id"])
    break

print("Commande valide :", commande["id"])

else:
    print("Toutes les commandes sont valides")

```

9 Imbrication contrôlée

```

"""
Table de multiplication 1 à 3.
Attention : éviter imbrication excessive.
Deux boucles imbriquées = complexité  $O(n^2)$ .
"""

for i in range(1, 4):
    for j in range(1, 4):
        print(i, "x", j, "=", i * j)

```

Pièges fréquents

Modifier une liste pendant le parcours

```

"""
Modifier une liste pendant son parcours peut provoquer
des sauts d'éléments silencieux.
Solution : parcourir une copie avec [:].
"""

liste = [1, 2, 3, 4]

for n in liste[:]:    # copie
    if n % 2 == 0:
        liste.remove(n)

print(liste)

```

Variable de boucle accessible après la boucle

```
"""
En Python, la variable de boucle reste définie
après la fin du for.
"""
```

```
for i in range(3):
    pass

print("Dernière valeur de i :", i)    # 2
```

1 1 Comparaison while vs for

Utiliser while quand :

- La condition dépend d'un état externe
- Le nombre d'itérations est inconnu

Utiliser for quand :

- On parcourt une collection
 - Le nombre d'itérations est déterminé
-

1 2 Ouverture — iter() et next()

```
"""
iter() crée un itérateur depuis un itérable.
next() récupère l'élément suivant manuellement.
Ce mécanisme est la base des itérateurs personnalisés
qui seront approfondis en Partie II.
"""
```

```
liste = [10, 20, 30]
it     = iter(liste)

print(next(it))    # 10
print(next(it))    # 20
print(next(it))    # 30
```

1 3 Mini-exercices de production

Exercice 1 — Prédiction

Prédire la sortie avant exécution :

```
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("terminé")

for i in range(5):
    if i == 3:
        continue
    print(i)
else:
    print("terminé")
```

Exercice 2 — Production

Écrire un programme qui :

1. Définit une liste de notes [12, 7, 15, 9, 18, 5]
2. Affiche chaque note avec son rang (en commençant à 1)
3. Affiche un message "Insuffisant" si la note est inférieure à 10
4. Calcule et affiche la moyenne finale
5. Affiche "Tous validés" ou "Des échecs présents" selon les résultats

Votre solution ici

Solution

```
"""
Parcours de notes avec enumerate, any et calcul de moyenne.
"""

notes = [12, 7, 15, 9, 18, 5]

for rang, note in enumerate(notes, start=1):
    mention = " ← Insuffisant" if note < 10 else ""
    print(f"Note {rang} : {note}{mention}")

moyenne = sum(notes) / len(notes)
print(f"\nMoyenne : {moyenne:.2f}")
```

```

if all(n >= 10 for n in notes):
    print("Tous validés")
else:
    print("Des échecs présents")

```

Exercice 3 — Débogage

Ce code contient 3 erreurs. Les identifier et les corriger.

```

liste = [1, 2, 3, 4, 5]

for n in liste:
    if n % 2 == 0:
        liste.remove(n)
print(liste)

noms = ["Alice", "Bob"]
ages = [25, 30, 35]
for nom, age in zip(noms, ages):
    print(nom, age)

i = 1
while i <= 5:
    print(i)

```

Solution

```

# Erreur 1 : modification de liste pendant parcours → utiliser copie [:]
# Erreur 2 : zip s'arrête au plus court → ages a 3 éléments mais noms 2
#                → pas d'erreur mais élément silencieusement ignoré
#                → aligner les deux listes
# Erreur 3 : i n'est jamais incrémenté → boucle infinie

```

```

liste = [1, 2, 3, 4, 5]

for n in liste[:]:
    if n % 2 == 0:
        liste.remove(n)
print(liste)

noms = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
for nom, age in zip(noms, ages):
    print(nom, age)

i = 1

```

```
while i <= 5:
    print(i)
    i += 1    # incrémentation ajoutée
```

Synthèse

- **while** répète tant qu'une condition est vraie — nombre d'itérations inconnu
 - **for** parcourt un itérable — nombre d'itérations déterminé
 - **break** interrompt la boucle courante uniquement
 - **continue** passe à l'itération suivante
 - **else** s'exécute uniquement si la boucle se termine sans **break**
 - **enumerate** fournit index + valeur sans compteur manuel
 - **zip** parcourt plusieurs itérables en parallèle
 - **any()** et **all()** remplacent avantageusement les boucles de vérification
 - Ne jamais modifier une liste pendant son parcours — utiliser **[:]**
 - Les compréhensions sont la forme idiomatique de **for** — approfondissement Partie II
-

20 Questions Intelligentes — Flux Itératifs

1) Différence fondamentale entre **while** et **for**

Énoncé

Quand choisir **while** plutôt que **for** ?

Solution

```
"""
Explication :
for est utilisé quand on connaît l'itérable ou le nombre d'itérations.
while est utilisé quand la répétition dépend d'une condition dynamique.
"""
```

```

# Cas déterminé
for i in range(3):
    print("for :", i)

# Cas conditionnel
compteur = 0
while compteur < 3:
    print("while :", compteur)
    compteur += 1

```

2) Pourquoi une boucle while peut devenir infinie ?

Énoncé

Identifier l'erreur logique.

Solution

```

"""
Explication :
Si la variable contrôlant la condition n'est jamais modifiée,
la condition reste toujours vraie.
Toujours vérifier que la condition de sortie peut être atteinte.
"""

i = 1

while i <= 3:
    print(i)
    i += 1    # sans cette ligne → boucle infinie

```

3) break interrompt quelle boucle ?

Énoncé

Dans une boucle imbriquée, laquelle est interrompue ?

Solution

```

"""
Explication :

```


break interrompt uniquement la boucle la plus interne.
La boucle externe continue son exécution normalement.

```
"""
```

```
for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print("i:", i, "j:", j)
```

Observation : la boucle externe continue.

4) continue saute quoi exactement ?

Énoncé

Que se passe-t-il après continue ?

Solution

```
"""
Explication :
continue ignore le reste du bloc courant
et passe directement à l'itération suivante.
"""
```

```
for i in range(5):
    if i == 2:
        continue
    print("Valeur :", i)
```

5) Clause else d'une boucle for

Énoncé

Quand le else s'exécute-t-il ?

Solution

```
"""
Explication :
Le else s'exécute uniquement si la boucle
se termine sans break.
"""
```

Si break est déclenché, le else est ignoré.

```
"""
```

```
# Sans break → else exécuté
for i in range(3):
    print(i)
else:
    print("Boucle terminée normalement")
```

```
# Avec break → else ignoré
for i in range(3):
    if i == 1:
        break
else:
    print("Jamais affiché")
```

6) Pourquoi enumerate est préférable ?

Énoncé

Comparer deux méthodes d'indexation.

Solution

```
"""
Explication :
enumerate évite la gestion manuelle d'un index.
Plus propre et moins risqué.
"""
```

```
noms = ["Alice", "Bob", "Charlie"]
```

```
# Version manuelle
i = 0
for nom in noms:
    print(i, nom)
    i += 1
```

```
# Version pythonique
for index, nom in enumerate(noms, start=1):
    print(index, nom)
```

7) Modification d'une liste pendant itération

Énoncé

Pourquoi ce code est dangereux ?

Solution

```
"""
Explication :
Modifier une liste pendant son parcours
peut provoquer des sauts d'éléments silencieux.
Solution : parcourir une copie avec [:].
"""

liste = [1, 2, 3, 4]

for n in liste[:]:
    if n % 2 == 0:
        liste.remove(n)

print(liste)
```

8) range stop exclusif

Énoncé

Pourquoi range(5) s'arrête à 4 ?

Solution

```
"""
Explication :
range(stop) génère de 0 à stop-1.
La borne stop est toujours exclusive.
Cohérent avec le slicing Python.
"""

for i in range(5):
    print(i)    # 0, 1, 2, 3, 4
```

9) range avec pas négatif

Énoncé

Comment compter à rebours ?

Solution

```
"""
Explication :
Un pas négatif inverse la direction.
start doit être supérieur à stop.
"""

for i in range(5, 0, -1):
    print(i)      # 5, 4, 3, 2, 1
```

10) zip pour itérer en parallèle

Énoncé

Comment parcourir deux listes simultanément ?

Solution

```
"""
Explication :
zip associe les éléments de même position.
S'arrête au plus court des itérables.
"""

noms = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 22]

for nom, age in zip(noms, ages):
    print(f"{nom} : {age} ans")

# Construction de dictionnaire
profil = dict(zip(noms, ages))
print(profil)
```

11) Court-circuit dans while

Énoncé

Pourquoi l'ordre des conditions est important ?

Solution

```
"""
Explication :
Les conditions sont évaluées de gauche à droite.
Placer la garde en premier évite une IndexError.
"""

liste = [1, 2, 3]
index = 0

while index < len(liste) and liste[index] != 3:
    index += 1

print("Index trouvé :", index)
```

12) Boucle sur dictionnaire

Énoncé

Différence entre for d et for d.items() ?

Solution

```
"""
Explication :
for d parcourt les clés uniquement.
items() fournit les paires clé + valeur.
values() fournit les valeurs uniquement.
"""

data = {"a": 1, "b": 2}

for cle in data:
    print("Clé :", cle)

for cle, valeur in data.items():
    print("Clé :", cle, "Valeur :", valeur)
```

```
for valeur in data.values():
    print("Valeur :", valeur)
```

13) Variable de boucle hors scope

Énoncé

Pourquoi `i` reste accessible après la boucle ?

Solution

```
"""
Explication :
En Python, la variable de boucle
reste définie après la fin du for.
Différent de certains autres langages.
"""

for i in range(3):
    pass

print("Dernière valeur de i :", i)    # 2
```

14) Utilité du `else` dans recherche

Énoncé

Pourquoi éviter une variable drapeau ?

Solution

```
"""
Explication :
La clause else évite l'usage d'un booléen externe.
Plus expressif et plus pythonique.
"""

liste = [1, 3, 5]

# Avec drapeau - à éviter
trouve = False
```

```

for n in liste:
    if n == 4:
        trouve = True
        break
if not trouve:
    print("Non trouvé")

# Avec else - pythonique
for n in liste:
    if n == 4:
        print("Trouvé")
        break
else:
    print("Non trouvé")

```

15) while vs input

Énoncé

Créer un menu interactif robuste.

Solution

```

"""
Boucle interactive contrôlée.
La boucle continue jusqu'à ce que l'utilisateur choisisse de quitter.
"""

while True:
    choix = input("1-Continuer 2-Quitter : ")

    if choix == "2":
        print("Au revoir")
        break
    elif choix == "1":
        print("Action exécutée")
    else:
        print("Choix invalide")

```

16) Double boucle et complexité

Énoncé

Quel est le coût d'une double boucle ?

Solution

```
"""
Explication :
Deux boucles imbriquées → complexité quadratique  $O(n^2)$ .
Pour  $n=10$  → 100 itérations.
Pour  $n=1000$  → 1 000 000 itérations.
Minimiser les imbrications sur de grands volumes.
"""

n = 3

for i in range(n):
    for j in range(n):
        print(i, j)

print(f"Total itérations pour n={n} : {n*n}")
```

17) Utilisation de any()

Énoncé

Remplacer une boucle de vérification par any.

Solution

```
"""
Explication :
any retourne True si au moins un élément satisfait la condition.
Plus concis qu'une boucle with drapeau.
Court-circuit : s'arrête au premier True trouvé.
"""

notes = [12, 15, 8, 17]

# Version boucle
echec = False
for n in notes:
```



```

    if n < 10:
        echec = True
        break

# Version any
if any(n < 10 for n in notes):
    print("Au moins une note insuffisante")

```

18) Utilisation de all()

Énoncé

Vérifier que toutes les conditions sont valides.

Solution

```

"""
Explication :
all retourne True si tous les éléments satisfont la condition.
Court-circuit : s'arrête au premier False trouvé.
"""

notes = [12, 15, 18]

# Version boucle
tous_valides = True
for n in notes:
    if n < 10:
        tous_valides = False
        break

# Version all
if all(n >= 10 for n in notes):
    print("Tous validés")

```

19) Performance : range vs liste

Énoncé

Pourquoi range est plus léger qu'une liste ?

Solution

```
"""
Explication :
range génère les valeurs à la volée sans créer une liste complète.
La mémoire utilisée est identique pour range(10) et range(10_000_000).
"""

import sys

r = range(1_000_000)
l = list(range(1_000_000))

print("Taille range :", sys.getsizeof(r), "octets")
print("Taille liste :", sys.getsizeof(l), "octets")
```

20) Refactorisation d'une boucle complexe

Énoncé

Simplifier un contrôle imbriqué.

Solution

```
"""
Explication :
La garde initiale (continue sur condition négative)
évite l'imbrication et améliore la lisibilité.
"""

utilisateurs = [
    {"nom": "Alice", "actif": True},
    {"nom": "Bob", "actif": False},
    {"nom": "Carol", "actif": True},
]

# Version imbriquée
for user in utilisateurs:
    if user["actif"]:
        print("Traitement :", user["nom"])

# Version avec garde
for user in utilisateurs:
    if not user["actif"]:
```

```
        continue  
    print("Traitement :", user["nom"])
```